

SQL Interview Preparation — Exact Chat Version

The following reproduces the previous chat answer in full; no substantive explanatory text, examples, or queries have been intentionally removed.

Absolutely. This is a **very good SQL interview list**. I'll explain all 35 in **simple interview language**, with **examples and queries**, so you can actually speak the answers in an interview rather than just memorize them.

I'll use mostly **MySQL syntax**.

☐ SQL INTERVIEW PREPARATION — 35 QUESTIONS

First, understand this sample database

We'll use these tables in many examples.

`employees`

emp_id	name	salary	dept_id	manager_id
1	Alice	50000	1	NULL
2	Bob	70000	1	1
3	Charlie	60000	1	1
4	David	40000	2	NULL
5	Emma	50000	2	4
6	Frank	70000	2	4

`departments`

dept_id	dept_name
1	IT
2	HR
3	Sales

1. What is SQL?

Simple answer

SQL stands for Structured Query Language.

It is used to **communicate with and manage relational databases**.

We use SQL to:

- Create tables
- Insert data
- Retrieve data
- Update data
- Delete data
- Control access to data

Example:

```
SELECT *  
FROM employees;
```

This retrieves all employees.

Interview answer

"SQL is a standard language used to interact with relational databases. It is mainly used for creating, reading, updating and deleting data."

2. WHERE vs HAVING

This is a **very common interview question**.

WHERE

`WHERE` filters **rows before grouping**.

```
SELECT *  
FROM employees  
WHERE salary > 50000;
```

It finds employees whose salary is greater than 50,000.

HAVING

`HAVING` filters **groups after GROUP BY**.

```
SELECT dept_id, AVG(salary)  
FROM employees  
GROUP BY dept_id  
HAVING AVG(salary) > 50000;
```

Here:

1. Employees are grouped by department.
2. Average salary is calculated.
3. Departments whose average salary is greater than 50,000 are selected.

Easy difference

WHERE → filters individual rows
HAVING → filters groups

Interview answer

"WHERE is used to filter rows before grouping, while HAVING is used to filter groups after GROUP BY."

3. DELETE vs DROP vs TRUNCATE

Very important.

DELETE

Removes rows from a table.

```
DELETE FROM employees  
WHERE dept_id = 2;
```

Only department 2 employees are deleted.

You can also delete everything:

```
DELETE FROM employees;
```

The table itself still exists.

TRUNCATE

Removes **all rows** from a table.

```
TRUNCATE TABLE employees;
```

The table structure remains.

You cannot normally use:

```
TRUNCATE TABLE employees  
WHERE dept_id = 2;
```

❑ Not allowed.

DROP

Deletes the **entire table**.

```
DROP TABLE employees;
```

Now the table itself is gone.

Remember

| Command | Removes rows | Removes table |

|---|---|---|

| DELETE | Yes | No |

| TRUNCATE | All rows | No |

| DROP | Everything | Yes |

Interview answer

"DELETE removes selected rows and supports WHERE. TRUNCATE removes all rows while keeping the table structure. DROP removes the complete table including its structure."

4. What is a Primary Key?

A **Primary Key** uniquely identifies each row in a table.

Example:

```
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY,  
  name VARCHAR(50),  
  salary INT  
);
```

Here `emp\_id` is the primary key.

It cannot contain:

```
NULL
```

and cannot contain duplicate values.

Example:

```
emp_id
-----
1
2
3
```

□ This is not allowed:

```
1
1
2
```

Interview answer

"A primary key is a column or combination of columns that uniquely identifies every row. It cannot contain NULL values or duplicate values."

5. What is a Foreign Key?

A Foreign Key creates a relationship between two tables.

Example:

```
CREATE TABLE departments (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50)
);
```

Then:

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(50),
  dept_id INT,
  FOREIGN KEY (dept_id)
  REFERENCES departments(dept_id)
);
```

Here:

```
departments.dept_id
↑
|
employees.dept_id
```

`employees.dept\_id` is the foreign key.

Why?

It prevents invalid department IDs.

For example, if departments only have:

```
1
2
3
```

you cannot insert:

```
dept_id = 10
```

if the foreign-key constraint prevents it.

Interview answer

"A foreign key is used to establish a relationship between two tables and maintain referential integrity."

6. CHAR vs VARCHAR

Both store strings.

CHAR

Fixed length.

```
CHAR(10)
```

If you store:

```
"Bob"
```

the field has a fixed size.

VARCHAR

Variable length.

```
VARCHAR(10)
```

It stores variable-length strings.

Example

```
name VARCHAR(100)
```

is generally suitable for names.

Easy difference

```
CHAR → fixed length  
VARCHAR → variable length
```

Interview answer

"CHAR is used for fixed-length strings, while VARCHAR is used for variable-length strings. VARCHAR is generally more space-efficient for strings whose lengths vary."

7. What is NULL?

`NULL` means **missing, unknown, or unavailable value**.

It does NOT mean:

```
0
```

and it does NOT mean:

```
"
```

Example:

```
name manager_id  
Alice NULL  
Bob 1
```

Alice has no manager recorded.

To check NULL:

```
SELECT *  
FROM employees  
WHERE manager_id IS NULL;
```

❑ Don't write:

```
WHERE manager_id = NULL;
```

Use:

```
IS NULL
```

or:

```
IS NOT NULL
```

Interview answer

"NULL represents a missing or unknown value. We use IS NULL or IS NOT NULL to check for NULL values."

8. UNION vs UNION ALL

Suppose:

```
SELECT name FROM employees1
UNION
SELECT name FROM employees2;
```

`UNION` removes duplicates.

UNION ALL

```
SELECT name FROM employees1
UNION ALL
SELECT name FROM employees2;
```

`UNION ALL` keeps duplicates.

Example:

Table 1:
Alice
Bob

Table 2:
Bob
Charlie

UNION

Alice
Bob
Charlie

UNION ALL

Alice
Bob
Bob
Charlie

Interview answer

"UNION combines result sets and removes duplicate rows, while UNION ALL combines result sets without removing duplicates. UNION ALL is generally faster because it doesn't perform duplicate elimination."

9. What are SQL Constraints?

Constraints are **rules applied to table columns**.

Common constraints:

PRIMARY KEY

id INT PRIMARY KEY

NOT NULL

name VARCHAR(50) NOT NULL

UNIQUE

email VARCHAR(100) UNIQUE

DEFAULT

country VARCHAR(30) DEFAULT 'India'

CHECK

salary INT CHECK (salary > 0)

FOREIGN KEY

FOREIGN KEY (dept_id)
REFERENCES departments(dept_id)

Interview answer

"Constraints are rules used to maintain data accuracy and integrity in a database."

10. DISTINCT vs GROUP BY

DISTINCT

Used to remove duplicate values.

```
SELECT DISTINCT dept_id  
FROM employees;
```

Result:

```
1  
2
```

GROUP BY

Used to create groups, usually for aggregate functions.

```
SELECT dept_id, COUNT(*)  
FROM employees  
GROUP BY dept_id;
```

Result:

```
dept_id COUNT  
1 3  
2 3
```

Easy difference

DISTINCT → remove duplicates
GROUP BY → create groups / aggregation

11. Explain SQL JOINS

JOINS are used to **combine data from multiple tables**.

Main types:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN

INNER JOIN

Returns matching records from both tables.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

LEFT JOIN

Returns **all rows from the left table** and matching rows from the right.

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Employees without a matching department will still appear.

RIGHT JOIN

Returns all rows from the right table.

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Interview answer

"A JOIN is used to combine rows from multiple tables based on a related column."

12. Find duplicate records

Suppose we have:

employees

```
name
----
Alice
Bob
Alice
Charlie
Bob
```

Find duplicate names:

```
SELECT name, COUNT(*) AS count
FROM employees
```

```
GROUP BY name
HAVING COUNT(*) > 1;
```

Output:

```
Alice 2
Bob 2
```

Remember

The pattern is:

```
GROUP BY column
HAVING COUNT(*) > 1
```

13. Remove duplicate records

This depends on what you mean by duplicate.

Suppose:

```
id name
1 Alice
2 Bob
3 Alice
4 Charlie
```

You want to keep one Alice.

One MySQL approach:

```
DELETE e1
FROM employees e1
JOIN employees e2
ON e1.name = e2.name
AND e1.id > e2.id;
```

This keeps the smaller `id` and deletes the duplicate.

Interview tip

If asked this question, first clarify:

"Which column or combination of columns defines a duplicate?"

Because duplicate can mean:

```
same name
```

or:

```
same name + salary + department
```

14. Find 2nd highest salary

A common interview question.

Method 1 — Subquery

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
  SELECT MAX(salary)
  FROM employees
);
```

Suppose:

70000
60000
50000
40000

First:

```
SELECT MAX(salary)
FROM employees;
```

→ `70000`

Then:

```
WHERE salary < 70000
```

Maximum of those:

60000

So second highest = `60000`.

To get employee name

```
SELECT name, salary
FROM employees
WHERE salary = (
  SELECT MAX(salary)
  FROM employees
  WHERE salary < (
    SELECT MAX(salary)
    FROM employees
  )
);
```

15. Find Nth highest salary

One common method uses `DENSE\_RANK()`.

For example, find 3rd highest:

```
SELECT name, salary
FROM (
  SELECT name,
  salary,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) x
WHERE rnk = 3;
```

Suppose salaries:

90000 → rank 1
80000 → rank 2
70000 → rank 3
60000 → rank 4

Result:

employees earning 70000

Why DENSE_RANK?

If salaries are:

90000
90000

80000
70000

DENSE_RANK gives:

90000 → 1
90000 → 1
80000 → 2
70000 → 3

16. Find highest salary by department

We need `GROUP BY`.

```
SELECT dept_id, MAX(salary) AS highest_salary
FROM employees
GROUP BY dept_id;
```

Output could be:

```
dept_id highest_salary
1 70000
2 70000
```

If you need employee names too

Use a window function:

```
SELECT name, salary, dept_id
FROM (
  SELECT name,
  salary,
  dept_id,
  RANK() OVER (
    PARTITION BY dept_id
    ORDER BY salary DESC
  ) AS rnk
  FROM employees
) x
WHERE rnk = 1;
```

This also returns multiple employees if there is a salary tie.

17. Find employees above average salary

First find average salary:

```
SELECT AVG(salary)
FROM employees;
```

Then use a subquery:

```
SELECT name, salary
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
);
```

Concept

Average salary = 56,666

Employees:
Alice 50000 □
Bob 70000 □
Charlie 60000 □

David 40000 □
Emma 50000 □
Frank 70000 □

18. Find employees without managers

If `manager\_id` contains NULL for employees without managers:

```
SELECT name  
FROM employees  
WHERE manager_id IS NULL;
```

Example:

Alice
David

Important

Don't use:

```
WHERE manager_id = NULL;
```

Use:

```
WHERE manager_id IS NULL;
```

19. Find departments with 5+ employees

Use:

```
SELECT dept_id, COUNT(*) AS employee_count  
FROM employees  
GROUP BY dept_id  
HAVING COUNT(*) >= 5;
```

Why HAVING?

Because we're filtering the **result of an aggregate**.

This would be wrong:

```
WHERE COUNT(*) >= 5
```

`COUNT()` is an aggregate, so use:

```
HAVING COUNT(*) >= 5
```

20. What are Subqueries?

A **subquery** is a query inside another query.

Example:

```
SELECT name, salary  
FROM employees  
WHERE salary > (  
  SELECT AVG(salary)  
  FROM employees  
);
```

The inner query:

```
SELECT AVG(salary)
FROM employees
```

finds the average.

The outer query finds employees above that average.

Interview answer

"A subquery is a query nested inside another query. It can be used in SELECT, FROM, WHERE, or HAVING depending on the requirement."

21. What are CTEs?

CTE = **Common Table Expression**.

It creates a temporary named result that can be used by the main query.

Syntax:

```
WITH avg_salary AS (
  SELECT AVG(salary) AS avg_sal
  FROM employees
)
SELECT *
FROM employees
WHERE salary > (
  SELECT avg_sal
  FROM avg_salary
);
```

Instead of putting everything into a nested subquery, CTE makes the query easier to read.

Interview answer

"A CTE is a temporary named result set defined using WITH. It improves readability and is useful for complex queries and recursive queries."

22. What are Window Functions?

Window functions perform calculations across related rows **without combining those rows into one row**.

Example:

```
SELECT name,
salary,
AVG(salary) OVER () AS average_salary
FROM employees;
```

Result:

```
name salary average_salary
Alice 50000 56666
Bob 70000 56666
Charlie 60000 56666
...
```

Notice every employee is still present.

Compare with:

```
SELECT AVG(salary)
FROM employees;
```

which returns only one row.

Common window functions

ROW_NUMBER()
RANK()
DENSE_RANK()
SUM()
AVG()
LAG()
LEAD()

23. RANK vs DENSE_RANK vs ROW_NUMBER

Extremely important.

Suppose salaries are:

100000
100000
90000
80000

ROW_NUMBER()

100000 → 1
100000 → 2
90000 → 3
80000 → 4

Every row gets a unique number.

RANK()

100000 → 1
100000 → 1
90000 → 3
80000 → 4

There is a gap after the tie.

DENSE_RANK()

100000 → 1
100000 → 1
90000 → 2
80000 → 3

No gap.

Easy memory trick

ROW_NUMBER → unique number
RANK → gaps after ties
DENSE_RANK → no gaps after ties

24. Find top 3 salaries per department

Use `DENSE_RANK()``:

```
SELECT name, salary, dept_id
FROM (
  SELECT name,
  salary,
  dept_id,
  DENSE_RANK() OVER (
    PARTITION BY dept_id
    ORDER BY salary DESC
```

```
) AS rnk
FROM employees
) x
WHERE rnk <= 3;
```

Important part

```
PARTITION BY dept_id
```

means:

Start ranking separately for each department.

Example:

```
IT:
70000 → 1
60000 → 2
50000 → 3
```

```
HR:
90000 → 1
80000 → 2
70000 → 3
```

25. Find a running total

Suppose sales:

```
date amount
Jan 1 100
Jan 2 200
Jan 3 300
```

Running total:

```
100
300
600
```

Query:

```
SELECT sale_date,
amount,
SUM(amount) OVER (
ORDER BY sale_date
) AS running_total
FROM sales;
```

Explanation

```
Jan 1 → 100
Jan 2 → 100 + 200 = 300
Jan 3 → 100 + 200 + 300 = 600
```

26. Find monthly sales

Suppose table:

```
sales
sale_date
amount
```

Use:

```
SELECT YEAR(sale_date) AS year,
MONTH(sale_date) AS month,
```

```
SUM(amount) AS total_sales
FROM sales
GROUP BY YEAR(sale_date), MONTH(sale_date)
ORDER BY year, month;
```

Output:

```
year month total_sales
2026 1 50000
2026 2 65000
2026 3 72000
```

27. Find year-over-year growth

Year-over-year means comparing a year with the **previous year**.

Suppose:

```
2024 → 100000
2025 → 120000
2026 → 150000
```

We can use `LAG()`.

```
WITH yearly_sales AS (
  SELECT YEAR(sale_date) AS year,
  SUM(amount) AS total_sales
  FROM sales
  GROUP BY YEAR(sale_date)
)
SELECT year,
total_sales,
LAG(total_sales) OVER (ORDER BY year) AS previous_year_sales,
(
  (total_sales - LAG(total_sales) OVER (ORDER BY year))
  / LAG(total_sales) OVER (ORDER BY year)
) * 100 AS growth_percentage
FROM yearly_sales;
```

Concept

```
2025 growth =
(2025 sales - 2024 sales) / 2024 sales × 100
```

28. Find consecutive login days

This is a more advanced interview question.

Suppose:

```
user_id login_date
1 2026-01-01
1 2026-01-02
1 2026-01-03
1 2026-01-05
```

The first three are consecutive.

One common approach uses `LAG()`:

```
SELECT user_id,
login_date,
LAG(login_date) OVER (
  PARTITION BY user_id
  ORDER BY login_date
) AS previous_login
FROM logins;
```

Then compare:

```
current_date = previous_date + 1 day
```

For example in MySQL:

```
SELECT user_id, login_date
FROM (
  SELECT user_id,
  login_date,
  LAG(login_date) OVER (
    PARTITION BY user_id
    ORDER BY login_date
  ) AS previous_login
  FROM logins
) x
WHERE DATEDIFF(login_date, previous_login) = 1;
```

This identifies logins that immediately follow the previous login.

29. Find inactive customers

Suppose we have:

customers

```
customer_id
1
2
3
```

orders

```
customer_id
1
1
3
```

Customer 2 has no orders.

Use:

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

Result:

```
Customer 2
```

Important pattern

```
LEFT JOIN
WHERE right_table.id IS NULL
```

is commonly used to find records with **no match**.

30. Find unmatched records

Suppose you want customers who have **no corresponding order**.

```
SELECT c.*
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
```

```
WHERE o.customer_id IS NULL;
```

This is an **anti-join pattern**.

You can also use `NOT EXISTS`:

```
SELECT c.*
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
);
```

Interview tip

If asked which approach you prefer, `NOT EXISTS` is often a clean choice when checking whether a related row exists.

31. Find each customer's 2nd purchase

This is a **window function** problem.

Suppose:

```
customer_id purchase_date
1 Jan 1
1 Jan 5
1 Jan 10
2 Jan 2
2 Jan 8
```

Use `ROW\_NUMBER()`:

```
SELECT customer_id, purchase_date
FROM (
  SELECT customer_id,
  purchase_date,
  ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY purchase_date
  ) AS rn
  FROM purchases
) x
WHERE rn = 2;
```

Result:

```
customer 1 → Jan 5
customer 2 → Jan 8
```

Why PARTITION BY?

It restarts numbering for each customer.

32. Find repeat customers

A repeat customer is someone who purchased **more than once**.

```
SELECT customer_id,
COUNT(*) AS purchase_count
FROM purchases
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Result:

```
customer_id purchase_count
1 3
2 2
```

Simple pattern:

```
GROUP BY customer
HAVING COUNT(*) > 1
```

33. Find the most-purchased product

Suppose:

```
product_id
1
1
1
2
2
3
```

Query:

```
SELECT product_id,
COUNT(*) AS purchase_count
FROM purchases
GROUP BY product_id
ORDER BY purchase_count DESC
LIMIT 1;
```

This gives the product with the highest purchase count.

If you want to handle ties

Use `DENSE\_RANK()`:

```
SELECT product_id, purchase_count
FROM (
SELECT product_id,
COUNT(*) AS purchase_count,
DENSE_RANK() OVER (
ORDER BY COUNT(*) DESC
) AS rnk
FROM purchases
GROUP BY product_id
) x
WHERE rnk = 1;
```

34. Find highest-paid employee per department

This is similar to #16, but here we need the **employee details**.

Use `RANK()`:

```
SELECT name, salary, dept_id
FROM (
SELECT name,
salary,
dept_id,
RANK() OVER (
PARTITION BY dept_id
ORDER BY salary DESC
) AS rnk
FROM employees
) x
WHERE rnk = 1;
```

Why `RANK()`?

Suppose department 1 has:

```
Bob 70000
Charlie 70000
Alice 50000
```

Both Bob and Charlie should be considered highest-paid.

`RANK()` returns both.

35. Find customers with no recent purchases

This question depends on what "**recent**" means.

For example, suppose recent means **last 30 days**.

We can use:

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN purchases p
ON c.customer_id = p.customer_id
AND p.purchase_date >= CURRENT_DATE - INTERVAL 30 DAY
WHERE p.customer_id IS NULL;
```

Important detail

Notice that the date condition is in the `ON` clause:

```
AND p.purchase_date >= CURRENT_DATE - INTERVAL 30 DAY
```

This means:

Find customers who don't have a purchase in the last 30 days.

This is different from simply finding customers who have **never purchased**.

☐ Most Important SQL Patterns for Interviews

Don't try to memorize all 35 queries separately. Learn these patterns.

1. Find duplicates

```
GROUP BY column
HAVING COUNT(*) > 1
```

2. Filter groups

```
GROUP BY column
HAVING aggregate_condition
```

3. Find maximum

```
SELECT MAX(salary)
FROM employees;
```

4. Find 2nd highest

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

5. Above average

```
WHERE salary > (SELECT AVG(salary) FROM employees)
```

6. No matching record

```
LEFT JOIN ...  
WHERE right_table.id IS NULL
```

or:

```
WHERE NOT EXISTS (...)
```

7. Ranking

```
DENSE_RANK() OVER (  
ORDER BY salary DESC  
)
```

8. Ranking within department

```
DENSE_RANK() OVER (  
PARTITION BY dept_id  
ORDER BY salary DESC  
)
```

9. Running total

```
SUM(amount) OVER (  
ORDER BY date  
)
```

10. Previous row

```
LAG(column) OVER (  
ORDER BY date  
)
```

11. Next row

```
LEAD(column) OVER (  
ORDER BY date  
)
```

□ How to Answer SQL Questions in an Interview

Since you're preparing for interviews, **don't immediately start writing the query.**

Use this 3-step approach:

Step 1 — Explain the logic

For example, interviewer asks:

Find the second highest salary.

Say:

"First, I will find the maximum salary. Then I will exclude that salary and find the maximum salary from the remaining records."

Step 2 — Write the query

```
SELECT MAX(salary)  
FROM employees  
WHERE salary < (  
SELECT MAX(salary)  
FROM employees)
```

);

Step 3 — Explain the query

Say:

"The inner query finds the highest salary. The outer query finds the maximum salary that is less than that value, which gives me the second highest distinct salary."

That's much better than silently writing SQL.

□ 10 Questions I Would Practice First

For an entry-level/fresher SQL interview, I'd prioritize these:

1. **WHERE vs HAVING**
2. **DELETE vs TRUNCATE vs DROP**
3. **Primary Key vs Foreign Key**
4. **JOINS**
5. **Find duplicate records**
6. **Find 2nd highest salary**
7. **Find Nth highest salary**
8. **Employees above average salary**
9. **GROUP BY + HAVING**
10. **RANK vs DENSE_RANK vs ROW_NUMBER**

Then move to:

CTE
Window Functions
Top N per department
Running total
LAG / LEAD
Second purchase
Repeat customers
Year-over-year growth

These latter questions are where interviewers can test whether you **actually understand SQL**, rather than just knowing basic ``SELECT``, ``WHERE``, and ``JOIN``.